# A Domain-Specific Language for Market Monitoring: From Domain Logic to SAS Code



SEASUG 2026; Malvern, PA

Eric C. Grasby, MSIQ
October 22-24, 2026

UA LITTLE ROCK

EXPECT MORE

# Thank you: Co-Authors!



Dr. Maria Gingras



ResearchGate Dr. Daniel Berleant




# Problem Landscape

# Unpacking the Title



# Domain-Specific Languages

A class of programming languages that are distinct from *general purpose programming languages* (GPLs). Domain-specific languages (DSLs) are programming languages that are:

1. Narrow in scope
2. High-level (abstract)
3. Typically intended for **non-developers**




*(Basically anything)*QL







DSLs ? GPLs

# Market Monitoring

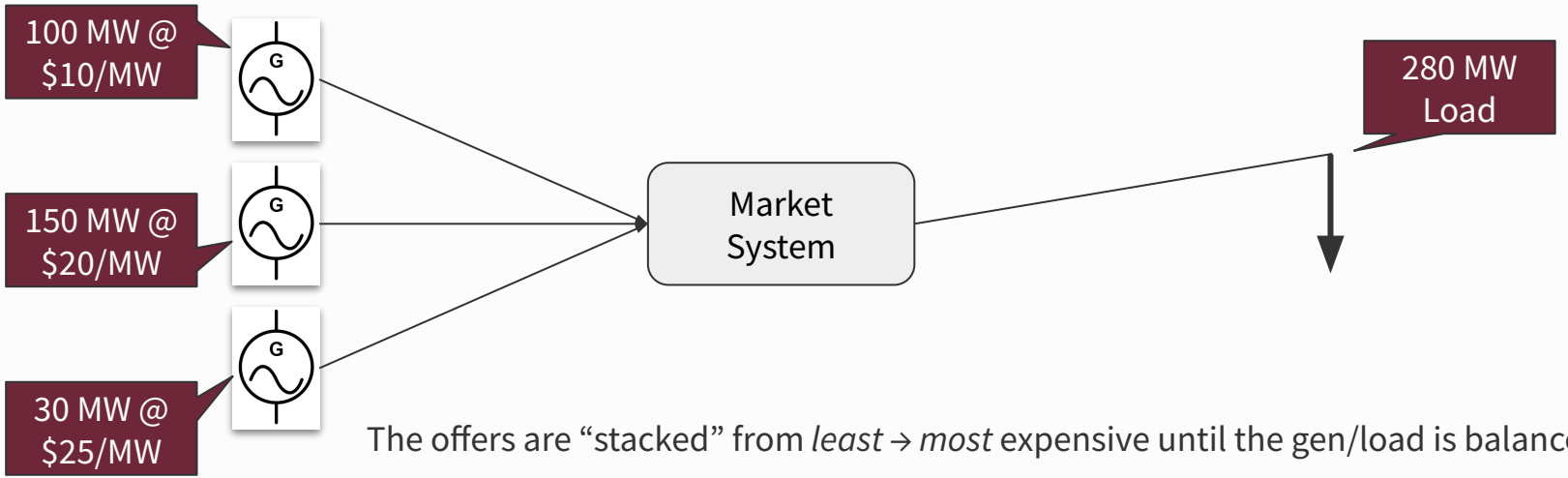
- Oversight organizations that provide market surveillance and policy analysis services to ensure that **organized, wholesale, electric markets remain fair and competitive**
- Required for all organized electric markets within the United States[1], but also exist outside of the US (APAC, EMEA, etc.)
- In the US, market monitors fall under the jurisdiction of the Federal Energy Regulatory Commission (FERC)




[1]https://www.ecfr.gov/current/title-18/chapter-I/subchapter-A/part-1c

# So What is a Wholesale Electricity Market?

A highly connected system that optimizes the price of bulk electricity subject to system constraints. Wholesale electricity prices, in the US, are both **marginal** and **location-dependent (spatial).**

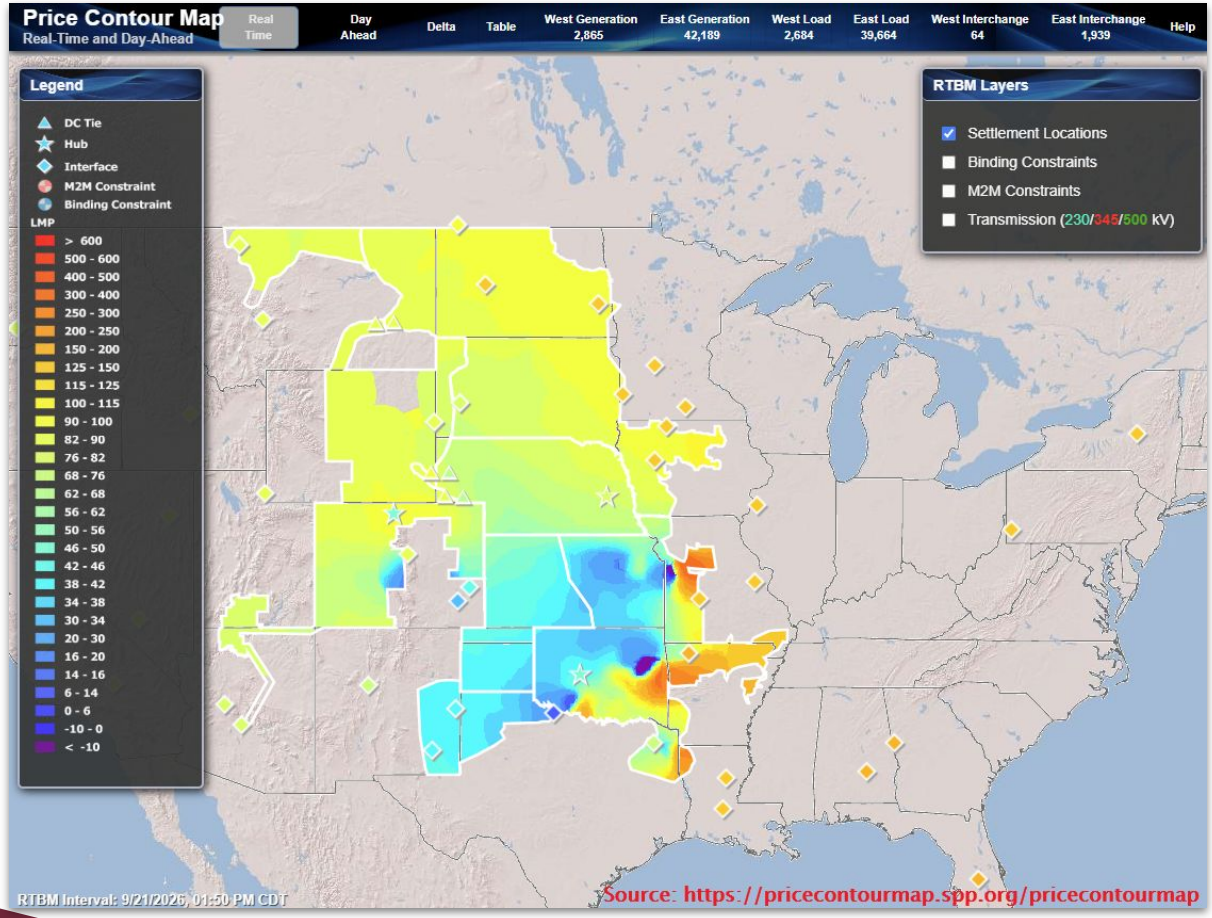



The offers are "stacked" from *least* → *most* expensive until the gen/load is balanced

100 MW + 150 MW + 30 MW = **280 MW**

Locational Marginal Price (LMP), for this market interval, is set at $25/MW

# SPP Market Territory




The marginal pricing example from earlier is set at hundreds of different pricing nodes ("PNodes") in the SPP footprint.

Prices approximated as color gradients where:

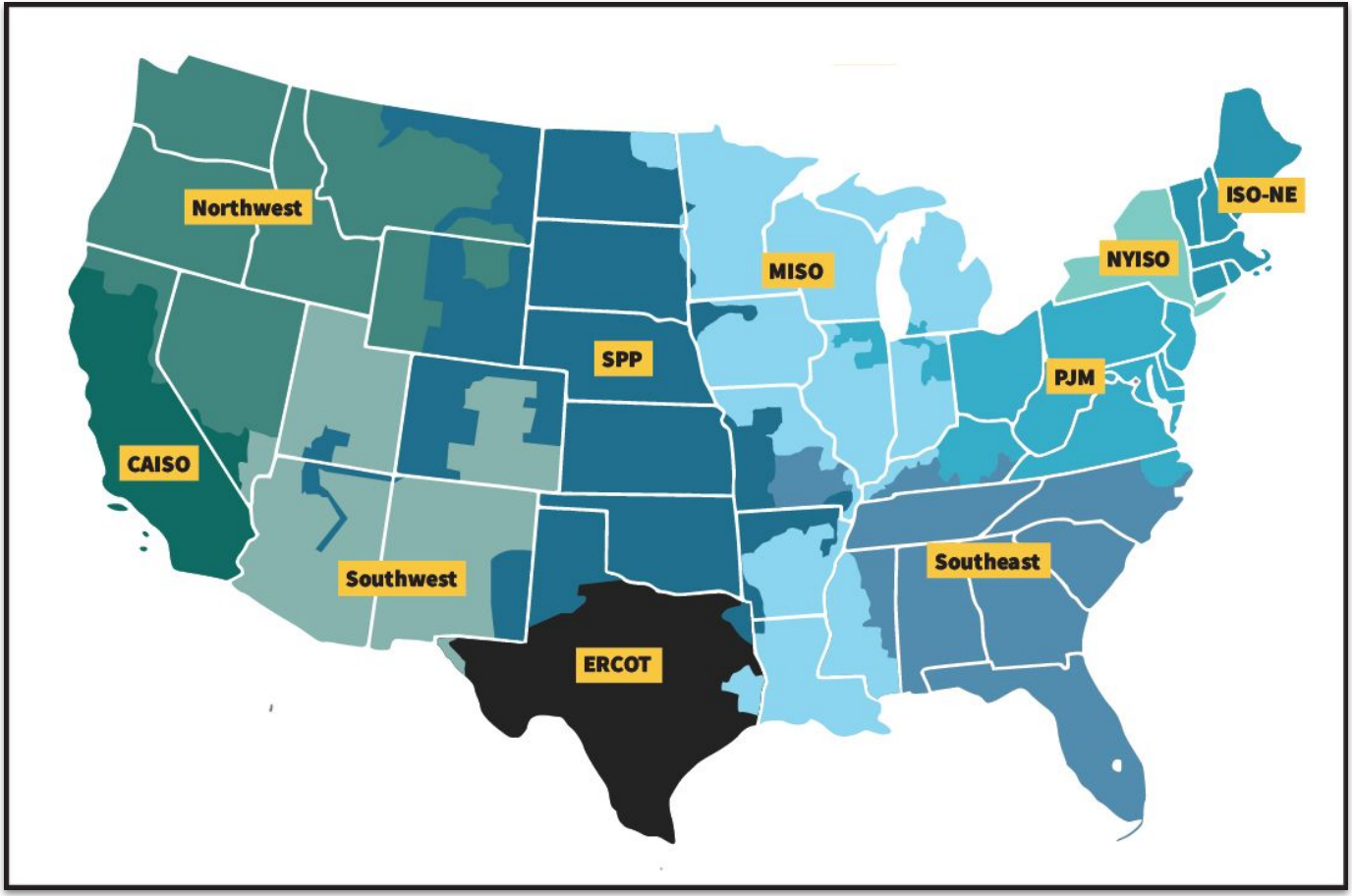
- Low prices are **purple**
- High prices are **red**

Under FERC Order 831:

- All energy offers are competitive up to $1000/MW.
- Any energy offer > $1000/MW is mitigated *to cost.*
- Marginal price shall not exceed $2000/MW

Source: https://pricecontourmap.spp.org/pricecontourmap

# Wholesale Electric Markets in the United States




Source: https://www.ferc.gov/electric-power-markets/




# Proposed Idea

# How is SAS involved?

- SAS is heavily used in market monitoring
  - Somewhere between a DSL and GPL (depending on your perspective)
  - Excellent for data processing, *but what if you're not a developer?*
- New analysts and/or engineers may:
  - Inherit old SAS code
  - Struggle with the complexity of electricity market data
  - "Recreate the wheel"

**<u>This is already really complex … how do you simplify it?</u>**

# MLang: Bring the logic closer to the analysts!

What if we used the DSL approach?

- Instead of starting with SAS – let's *end with* SAS
- Express market monitoring tasks in a *bespoke programming language*
- *Transpile* this new language *into* valid SAS code

Encapsulate the complex tasks as either SAS Macros, DATA step programs, or custom functions

- Let the transpiler determine which "chunk" of code to call
- Custom validations and warnings
- Alert the user about potential issues *prior to runtime*

# Speaking "Plainly"

- Electric markets need to be constantly monitored
- The data involved is complex and ever-changing
- Monitors wear "many hats"
- Is the solution to require market monitors to also be programmers? **No.**
- Make the "programming" more accessible to them? **Yes!**

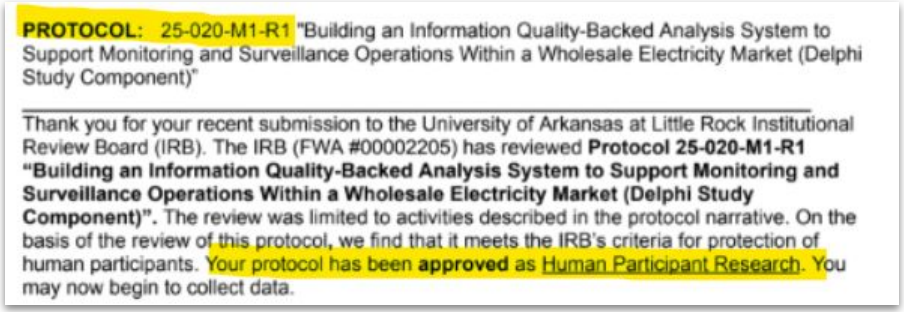
**"The goal is to empower the users to do meaningful work"**




# Study Design

# Where to Begin?

- Who would best know what a system to aid market monitors should look like?
  - The monitors themselves!
- Contacted the Energy Intermarket Surveillance Group (EISG)
- Requested to conduct a research study to understand workflow
- Delphi study (expert panel of anonymous participants)



**PROTOCOL:** 25-020-M1-R1 "Building an Information Quality-Backed Analysis System to Support Monitoring and Surveillance Operations Within a Wholesale Electricity Market (Delphi Study Component)"

Thank you for your recent submission to the University of Arkansas at Little Rock Institutional Review Board (IRB). The IRB (FWA #00002205) has reviewed **Protocol 25-020-M1-R1 "Building an Information Quality-Backed Analysis System to Support Monitoring and Surveillance Operations Within a Wholesale Electricity Market (Delphi Study Component)".** The review was limited to activities described in the protocol narrative. On the basis of the review of this protocol, we find that it meets the IRB's criteria for protection of human participants. Your protocol has been **approved** as Human Participant Research. You may now begin to collect data.

# Delphi Study

- 19 members of the EISG opted to participate
- 2 rounds of survey questions with a mid-round “recap” paper
  - Participants responded anonymously to questions and then received summaries
- Anonymity helps filter out bias and reduces “groupthink”
- Processed results became software requirements

Q5: “What are the primary data challenges that you face in market monitoring?”
Q7: “Explain the data sources that you read from and write to in the course of your normal work.”
Q41: “If you had a set of commands at your disposal that could perform often tedious tasks that you have to do in your role, what commands would you want?”

# Thematic Coding

Q5

| Theme | Number of Mentions |
|---|---|
| Data Quality and Literacy | 6 |
| Data Integration | 4 |
| Data Volume and Scalability | 4 |
| Metadata, Discovery, and Documentation | 4 |
| Infrastructure and Computing Resources | 2 |
| Analytics Lifecycle Management | 1 |
| Visualization Creation and Reporting | 1 |
| Data Governance | 1 |

Data Quality Checks

Q7

| Theme | Number of Mentions |
|---|---|
| Database | 15 |
| CSV | 9 |
| Excel | 4 |
| Vendor Data | 4 |
| Public Data | 4 |
| API | 4 |
| SAS | 2 |
| XML | 1 |

Import/Export Targets

Q41

| Theme | Number of Mentions |
|---|---|
| Market Surveillance and Monitoring | 7 |
| Data Exploration and Querying | 4 |
| Validation and Quality Assurance | 2 |
| Portfolio and Resource Analysis | 2 |
| Metadata and Domain Knowledge | 1 |
| Reporting and Automation | 1 |

Tagging

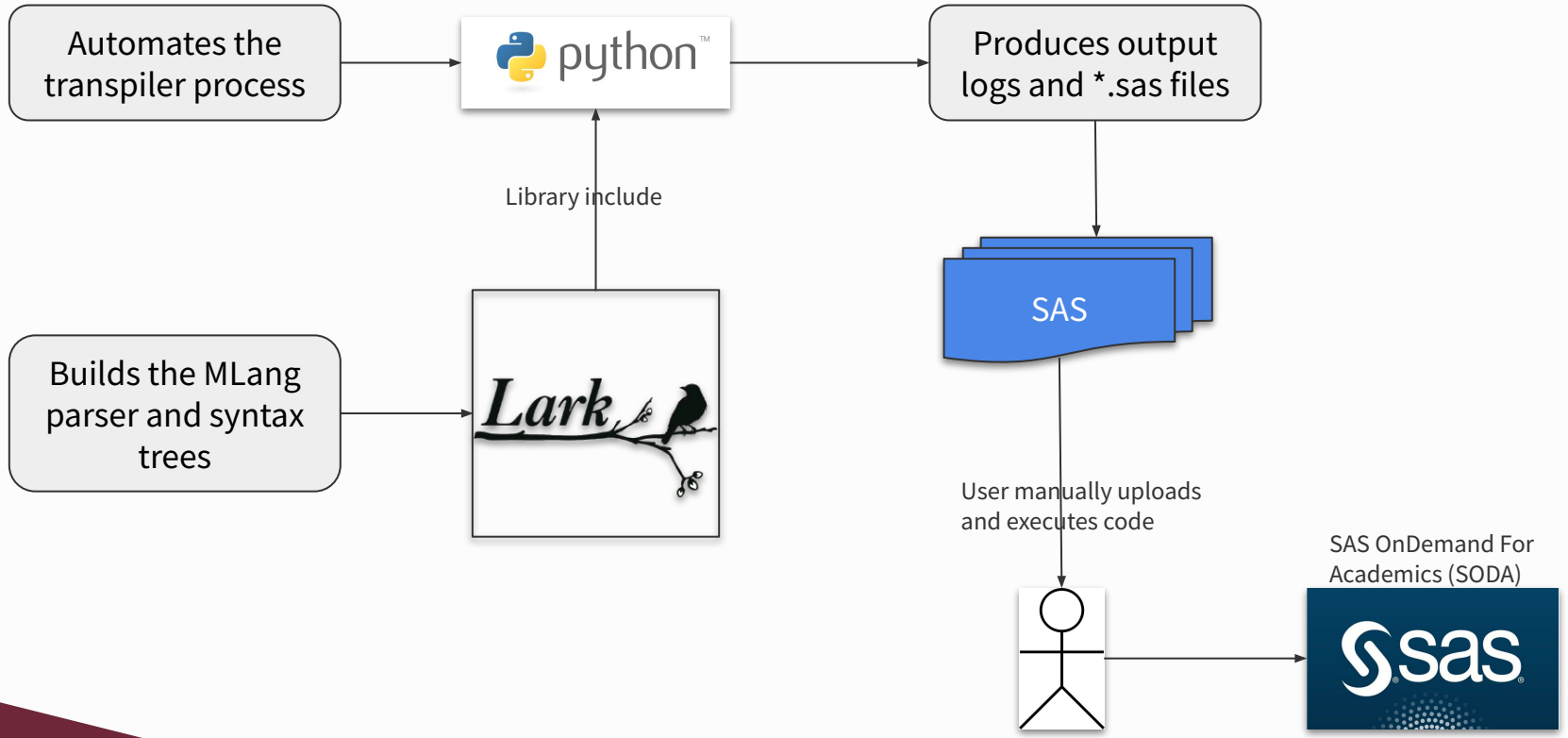
# Basic Requirements and Use Cases

| ID | Requirement | Passing Criteria |
|---|---|---|
| 1 | The language shall be declarative. A user can define what they want the analysis and/or output to be, and the transpiler generates code in the target language to accomplish the task. | The user can write a script in the language's declarative grammar and receives transpiled code in the target language. |
| 2 | The language, where possible, should infer metadata needed for data selection, transformations, and filter criteria from a metadata repository. | The user can use the language to define metadata (e.g. tags) and associate it with a data set. |
| 3 | The language shall, at a minimum, support database queries and CSV files as both read and write targets *(the leading 2 data sources mentioned by Delphi participants).* | A user can specify a database connection, CSV file paths, or both, and use these as sources and destinations for sourcing and sinking data. |
| 4 | The language shall allow the user to generate textual reports of pre-defined metrics (as *an example, analyzing resources by different dimensions, fuel-type, time period, capacity, etc.).* | The user can calculate a pre-defined metric for an operating period and view the report. |
| 5 | The language shall allow the user to generate simple visualizations of pre-defined metrics to support market reporting *(possible examples include generating fleet fuel-mix or capacity of generation on outage or locating generating facilities or constraints on a cartographic map).* | The user can create certain pre-defined visualizations that provide metrics for market reporting. |




# POC Implementation

# Technology Stack



# Grammar and MLang Source

```
start: statement+

statement: create_dataset_stmt
         | tag_stmt
         | search_stmt

// CREATE DATASET NAME FROM CSV("/path/to/file.csv");
create_dataset_stmt: "CREATE"i "DATASET"i IDENTIFIER "FROM"i source ";"

source: "CSV"i "(" STRING ")"

// TAG target WITH key = "value";
// TAG target WITH (key1 = "value1", key2 = "value2", );
tag_stmt: "TAG"i target "WITH"i tag_body ";"

target: IDENTIFIER ("." IDENTIFIER)?

tag_body: tag_pair
        | "(" tag_pair ("," tag_pair)* ","? ")"

tag_pair: IDENTIFIER "=" STRING

// SEARCH BY TAG(name) THEN ORDER BY ASC;
search_stmt: "SEARCH"i "BY"i "TAG"i "(" IDENTIFIER ")" order_clause? ";"

order_clause: "THEN"i "ORDER"i "BY"i direction

direction: "ASC"i -> asc
         | "DESC"i -> desc

IDENTIFIER: /[A-Za-z_][A-Za-z0-9_]*/

STRING: /"(?:[^"\\]|\\.)*"/

LANG_COMMENT: /\/\*[\s\S]*?\*\//

// Lark Imports
%import common.WS

%ignore WS

%import common.CPP_COMMENT

%ignore CPP_COMMENT

%ignore LANG_COMMENT
```

```
/*  ================================================================
    Author: Eric C. Grasby
    8/5/2026
    ================================================================
*/

/* Create new dataset: power plants (Source: EIA)*/
CREATE DATASET PLANTS FROM CSV("../data/EIA_860_PLANTS.csv");

/* Tag "EIA_SOURCE_PLANTS" with metadata */
TAG PLANTS WITH SOURCE_ORG = "Energy Information Authority (EIA)";

TAG PLANTS.UTILITY_ID WITH (PRIMARY_KEY = "True" , UNIQUE = "True", );

TAG PLANTS.PLANT_CODE WITH PRIMARY_KEY = "True";

/* Create new dataset: generating utilities (Source: EIA)*/
CREATE DATASET UTILITIES FROM CSV("../data/EIA_860_UTILITIES.csv");

/* Tag "EIA_SOURCE_UTILITIES" with metadata */
TAG UTILITIES WITH SOURCE_ORG = "Energy Information Authority (EIA)";

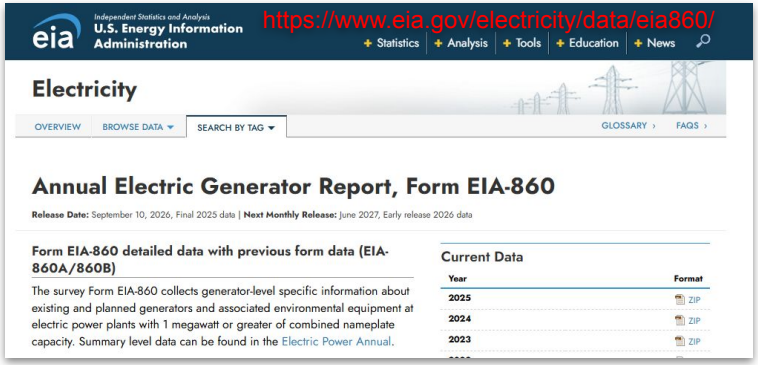
tag UTILITIES.utility_id with (primary_key = "true", unique = "true"); /*
.mlang is case insensitive! */

/* Create new dataset: generators (Source: EIA) */
CREATE DATASET GENERATORS FROM CSV("../data/EIA_860_GENERATORS.csv");

/* Search for datasets / fields that are tagged with the "SOURCE_ORG" tag */
SEARCH BY TAG(SOURCE_ORG) THEN ORDER BY ASC;

/* line 29 would also have been valid to say: `SEARCH BY TAG(SOURCE_ORG);`
   .mlang would have implicitly ordered results ascending
*/
```

# Test Data




EIA Form 860 is a general purpose, publicly available, data set that informs market monitors of generator physical parameters.

The raw data set is messy, non-standardized, and is a good test set to exhibit some of the functionality of MLang.

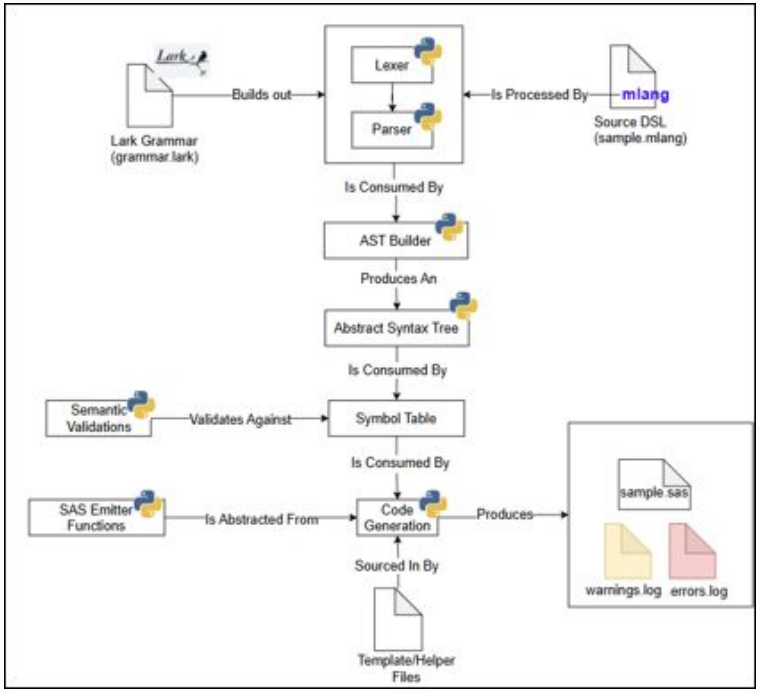
2024 Form EIA-860 Data - Schedule 3, 'Generator Data' (Operable Units Only)

| Utility ID | Utility Name | Plant Code | Plant Name | State | County | Generator ID | Technology | Prime Mover | Unit Code | Ownership | Duct Burners | Can Bypass Heat Recovery Steam Generator? | RTO/ISO LMP Node Designation |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 63560 | Sand Point Generating, LLC | 1 | Sand Point | AK | Aleutians East | 1 | Petroleum Liquids | IC | | S | X | X | |
| 63560 | Sand Point Generating, LLC | 1 | Sand Point | AK | Aleutians East | 2 | Petroleum Liquids | IC | | S | X | X | |
| 63560 | Sand Point Generating, LLC | 1 | Sand Point | AK | Aleutians East | 3 | Petroleum Liquids | IC | | S | X | X | |
| 63560 | Sand Point Generating, LLC | 1 | Sand Point | AK | Aleutians East | 5.1 | Petroleum Liquids | IC | | S | X | X | |
| 63560 | Sand Point Generating, LLC | 1 | Sand Point | AK | Aleutians East | WT1 | Onshore Wind Turbine | WT | | S | X | X | |
| 63560 | Sand Point Generating, LLC | 1 | Sand Point | AK | Aleutians East | WT2 | Onshore Wind Turbine | WT | | S | X | X | |

2024 Form EIA-860 Data - Schedule 2, 'Plant Data'

| Utility ID | Utility Name | Plant Code | Plant Name | Street Address | City | State | Zip | County | Latitude | Longitude | NERC Region | Balancing Authority Code | Balancing Authority Name | Name of Water Source |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 63560 | Sand Point Generating, LLC | 1 | Sand Point | 100 Power Plant Way | Point | AK | 99661 | Aleutians East | 55.339722 | -160.497222 | | | | |
| 195 | Alabama Power Co | 2 | Bankhead Dam | 19001 Lock 17 Road | Northport | AL | 35476 | Tuscaloosa | 33.458665 | -87.356823 | SERC | SOCO | Company | Black Warrior River |
| 195 | Alabama Power Co | 3 | Barry | North Highway 43 | Bucks | AL | 36512 | Mobile | 31.006900 | -88.010300 | SERC | SOCO | Company | Mobile River |
| 195 | Alabama Power Co | 4 | Walter Bouldin Dam | 750 Bouldin Dam Road | a | AL | 36092 | Elmore | 32.583889 | -86.283056 | SERC | SOCO | Company | Coosa River |
| 195 | Alabama Power Co | 7 | Gadsden | 1000 Goodyear Ave | Gadsden | AL | 35903 | Etowah | 34.012800 | -85.970800 | SERC | SOCO | Company | Coosa River |

# Transpiler




Transpiler (translate + compile) is a **source to source compiler** program.

High-level lang (A) → Transpile → High-level lang (B)

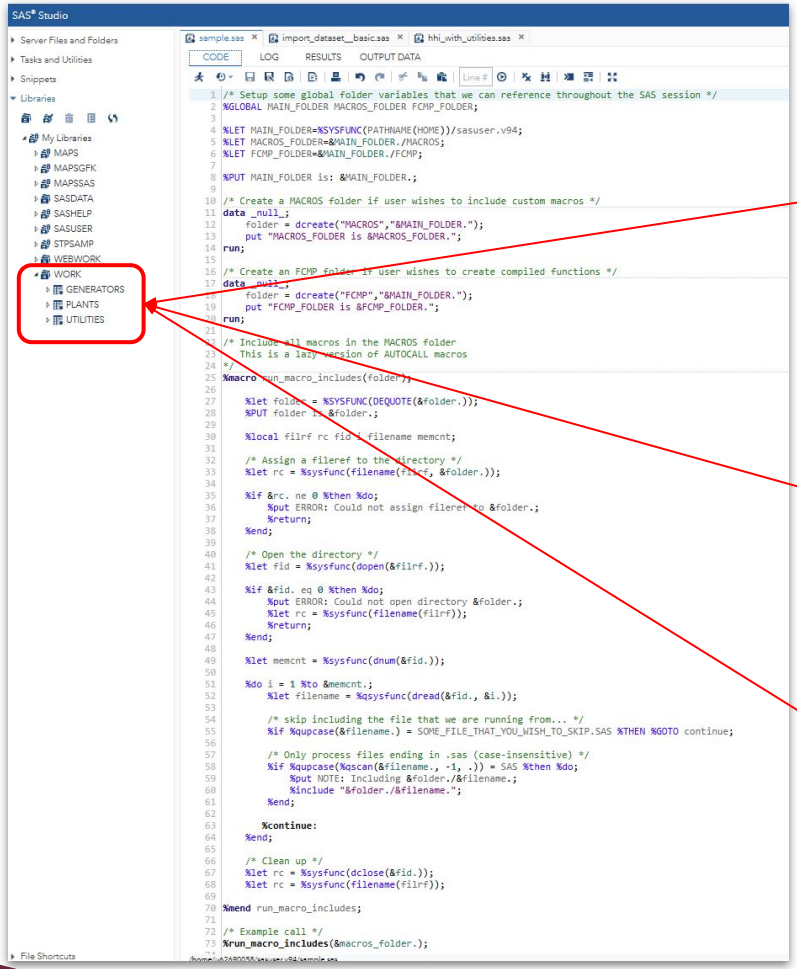



Igor Khorlo (Syneos Health) has presented on related topics before (SAS Global Forum 2018, 2019):

- Automating Migration from the SAS Macro Language to Lua
- SASLint: A SAS Program Checker

We are extending this concept, but parsing existing logic and translating it *into SAS code.*

# Sample Program Execution




```
/* Create Dataset: PLANTS (invokes import_dataset__basic.sas) */
%import_dataset__basic(source=&MAIN_FOLDER./DATA/EIA_860_PLANTS.csv,out_name=PLANTS,source_format=CSV);

PROC DATASETS library=WORK NOLIST;
    MODIFY PLANTS;
    XATTR SET DS SOURCE_ORG='Energy Information Authority (EIA)';
QUIT;
PROC DATASETS LIBRARY=WORK NOLIST;
    MODIFY PLANTS;
    XATTR ADD VAR UTILITY_ID (
                                    PRIMARY_KEY='True'
UNIQUE='True'
                                );
QUIT;
PROC DATASETS LIBRARY=WORK NOLIST;
    MODIFY PLANTS;
    XATTR ADD VAR PLANT_CODE (
                                    PRIMARY_KEY='True'
                                );
QUIT;

/* Create Dataset: UTILITIES (invokes import_dataset__basic.sas) */
%import_dataset__basic(source=&MAIN_FOLDER./DATA/EIA_860_UTILITIES.csv
,out_name=UTILITIES
,source_format=CSV);

PROC DATASETS library=WORK NOLIST;
    MODIFY UTILITIES;
    XATTR SET DS SOURCE_ORG='Energy Information Authority (EIA)';
QUIT;
PROC DATASETS LIBRARY=WORK NOLIST;
    MODIFY UTILITIES;
    XATTR ADD VAR utility_id (
                                    primary_key='true'
unique='true'
                                );
QUIT;

/* Create Dataset: GENERATORS (invokes: import_dataset__basic.sas) */
%import_dataset__basic(source=&MAIN_FOLDER./DATA/EIA_860_GENERATORS.csv
,out_name=GENERATORS
,source_format=CSV);
```

# Features

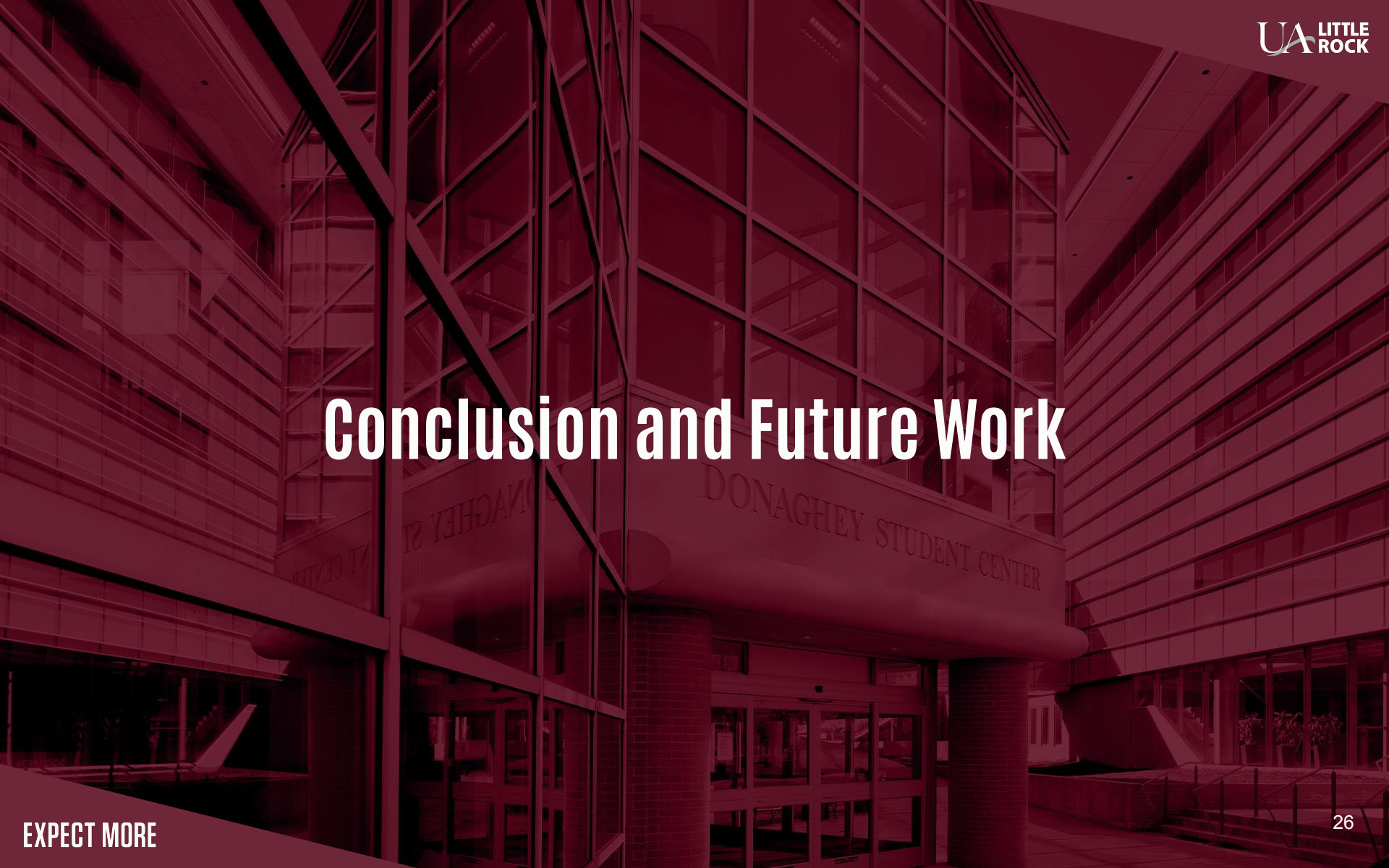
**Results of Tag Search: 09SEP2026:10:22:18**

| Library Name | Member Name | Column Name | Extended Attribute Name | Extended Attribute Value |
|---|---|---|---|---|
| WORK | PLANTS | | SOURCE_ORG | Energy Information Authority (EIA) |
| WORK | UTILITIES | | SOURCE_ORG | Energy Information Authority (EIA) |

```
[source] Field names 'FERC_EXEMPT_WHOLESALE_GENERATOR_STATUS' and
'FERC_EXEMPT_WHOLESALE_GENERATOR_DOCKET_NUMBER' in data set 'EIA_SOURCE_PLANTS' collide once truncated
to 32 characters
[source] Field names 'TRANSMISSION_OR_DISTRIBUTION_SYSTEM_OWNER' and
'TRANSMISSION_OR_DISTRIBUTION_SYSTEM_OWNER_ID' in data set 'EIA_SOURCE_PLANTS' collide once truncated
to 32 characters
[source] Field names 'TRANSMISSION_OR_DISTRIBUTION_SYSTEM_OWNER' and
'TRANSMISSION_OR_DISTRIBUTION_SYSTEM_OWNER_STATE' in data set 'EIA_SOURCE_PLANTS' collide once
truncated to 32 characters
```

```
[source-data-field-name-length] Field name 'PLANNED_NET_WINTER_CAPACITY_DERATE_(MW)' is 39 characters
long; SAS names must be 32 characters or fewer
[source-data-field-name-charset] Field name 'PLANNED_NET_WINTER_CAPACITY_DERATE_(MW)' is not a valid
SAS identifier (must start with a letter or underscore, and contain only letters, digits, and
underscores)
[source-data-field-name-length] Field name 'PLANNED_NEW_NAMEPLATE_CAPACITY_(MW)' is 35 characters
long; SAS names must be 32 characters or fewer
[source-data-field-name-charset] Field name 'PLANNED_NEW_NAMEPLATE_CAPACITY_(MW)' is not a valid SAS
identifier (must start with a letter or underscore, and contain only letters, digits, and underscores)
```

- Templated data imports (CSV)
- Data tagging (XATTR)
- Searching for tags (SASHELP.VXATTR)
- Basic data quality checks
  - Invalid chars in asset names?
  - Asset names > 32 chars?
  - Asset name collisions on truncation?
  - Asset names shadow existing SAS keywords?




# Conclusion and Future Work

# Limitations

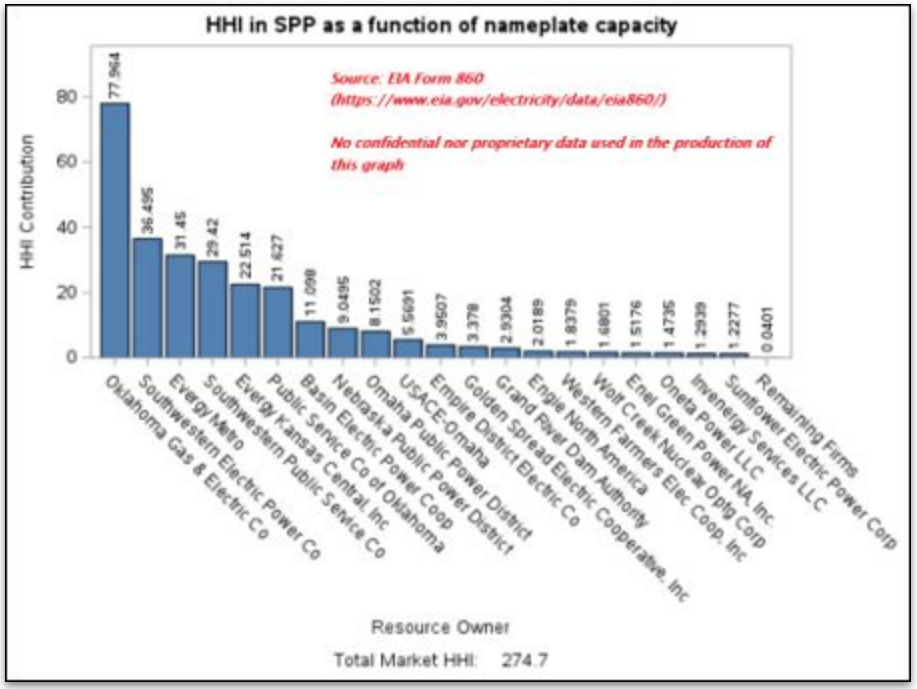
- The POC, currently, supports only a small number of functions
  - Data imports
  - Basic data quality checks
  - Data set and field tagging
  - Searching by tags
- The transpiler exists entirely as pre-processor directives
  - Use of the SAS code requires manual intervention
  - No syntax highlighting or LSP support
- Must be invoked from a command-line, which isn't very non-developer friendly
  - Run the transpiler program, read the output logs, copy the files, upload them to SODA, etc.

# Future Work

- Develop support for more varied and complex market monitoring use cases
  - Calculating market metrics (e.g. HHI) from language keywords
  - Map generation through SGMAP
- Including more complex data quality and semantic checks
- User experience (UX) support
  - Language Server Protocol (LSP)
  - Possibly SASLint[1] integration
- Automatic execution in a SAS runtime environment

[1] I. Khorlo, https://support.sas.com/resources/papers/proceedings18/2543-2018.pdf

# Market Metrics (Development in Progress)



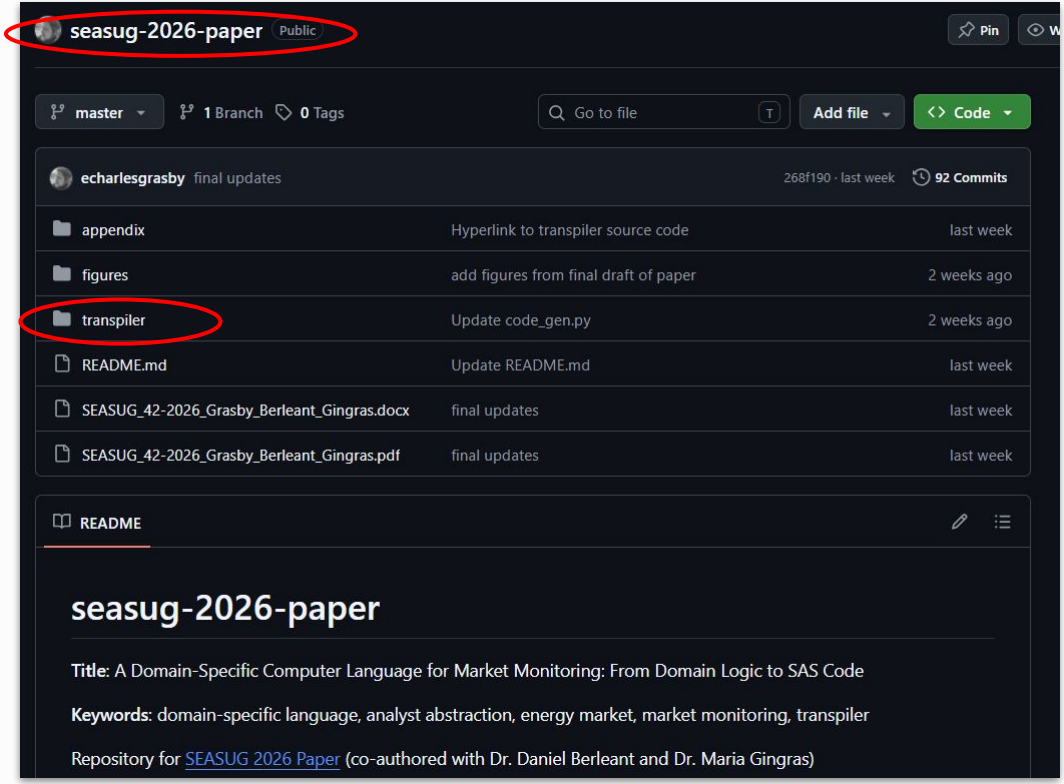

- Herfindahl-Hirschman Index (HHI) is a standard measure of market concentration
  - Inversely proportional to competitiveness
  - HHI < 1000, unconcentrated market
  - HHI > 10,000, monopoly
- Standard equation, but how you define the inputs to the equation can vary
- MLang will be able to compute HHI based upon user-defined input variable
  - This example uses “nameplate capacity”

$$HHI = \sum_{i=0}^{n} S_i^{\,2}$$

```
CREATE DATASET NAMEPLATE_HHI FROM HHI(PLANTS, NAMEPLATE_CAP);

CREATE OUTPUT GRAPH NAMEPLATE_HHI_GRAPH FROM NAMEPLATE_HHI WITH PROPERTIES { BAR_COLOR = “BLUE”, … };
```

# Paper and Source Code Repository




https://github.com/echarlesgrasby/seasug-2026-paper

# Follow Up Questions?

Feel free to contact the authors:


Eric Grasby, MSIQ (ecgrasby@ualr.edu)

Daniel Berleant, PhD (jdberleant@ualr.edu)

Maria Gingras, PhD (maria@ochotonaanalytics.com)




# Thank you: Co-Authors!



Dr. Maria Gingras



ResearchGate Dr. Daniel Berleant